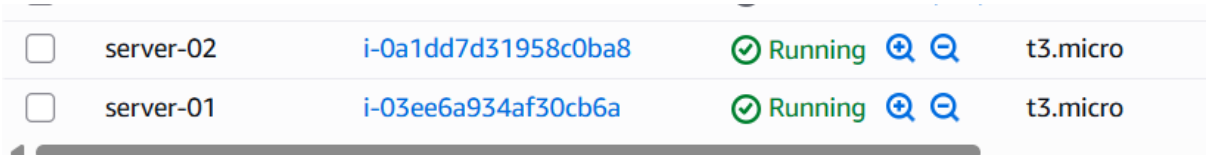
ABDUL JANEEB K

AWS Application Load Balancer (ALB) Practice Tasks

- **Basic Load Balancer**
- Launch two EC2 instances, install Apache, create a target group and ALB, then verify traffic distribution.

- Created two EC2 instances



- In both EC2-instance installed Apache(HTTPD) and their html files created

```
[ec2-user@ip-172-31-39-207 ~]$ sudo hostnamectl set-hostname server1
[ec2-user@ip-172-31-39-207 ~]$ hostname
server1
[ec2-user@ip-172-31-39-207 ~]$ exec bash
[ec2-user@server1 ~]$ sudo dnf install -y httpd
Amazon Linux 2023 Kernel Livepatch repository                   872 kB/s |  89 kB     00:00
Dependencies resolved.
================================================================================ Package
================================================================================Installing:
 httpd                  x86_64        2.4.69-1.amzn2023.0.2          amazonlinux          45 k
Installing dependencies:
 apr                    x86_64        1.7.5-1.amzn2023.0.4           amazonlinux         129 k
 apr-util               x86_64        1.6.5-1.amzn2023.0.1           amazonlinux         100 k
 apr-util-lmdb          x86_64        1.6.5-1.amzn2023.0.1           amazonlinux          15 k
 generic-logos-httpd    noarch        18.0.0-12.amzn2023.0.3         amazonlinux          19 k
 httpd-core             x86_64        2.4.69-1.amzn2023.0.2          amazonlinux         1.4 M
 httpd-filesystem       noarch        2.4.69-1.amzn2023.0.2          amazonlinux          12 k
 httpd-tools            x86_64        2.4.69-1.amzn2023.0.2          amazonlinux          79 k
 libbrotli              x86_64        1.1.0-1.amzn2023.0.2           amazonlinux         333 k
 mailcap                noarch        2.1.49-3.amzn2023.0.3          amazonlinux          33 k
Installing weak dependencies:
 apr-util-openssl       x86_64        1.6.5-1.amzn2023.0.1           amazonlinux          17 k
 mod_http2              x86_64        1:2.4.69-1.amzn2023.0.2        amazonlinux         130 k
 mod_lua                x86_64        2.4.69-1.amzn2023.0.2          amazonlinux          58 k

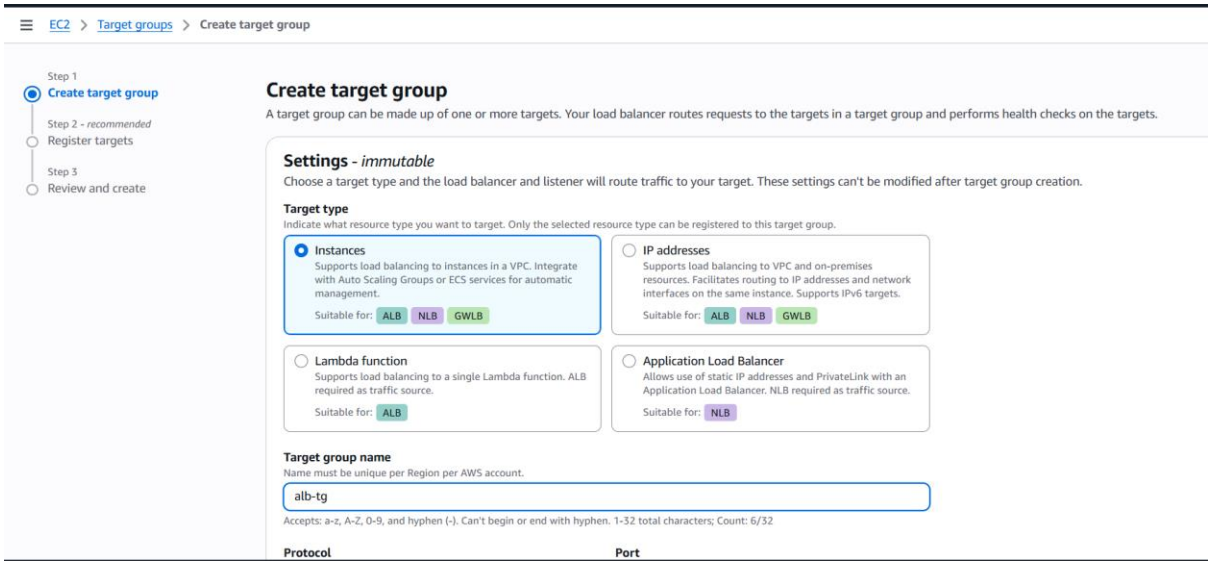
Transaction Summary
================================================================================Install  13 Packages

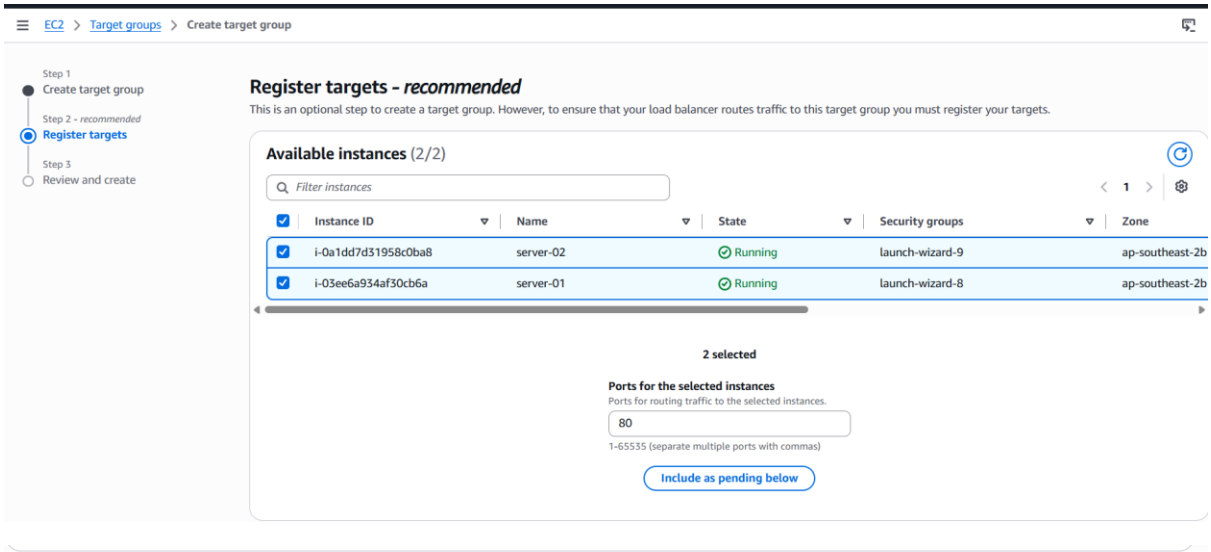
Total download size: 2.3 M
Installed size: 6.9 M
Downloading Packages:
(1/13): apr-1.7.5-1.amzn2023.0.4.x86_64.rpm                     4.2 MB/s | 129 kB     00:00
(2/13): apr-util-1.6.5-1.amzn2023.0.1.x86_64.rpm                3.0 MB/s | 100 kB     00:00
(3/13): apr-util-lmdb-1.6.5-1.amzn2023.0.1.x86_64.rpm           438 kB/s |  15 kB     00:00
(4/13): apr-util-openssl-1.6.5-1.amzn2023.0.1.x86_64.rpm        619 kB/s |  17 kB     00:00
(5/13): httpd-2.4.69-1.amzn2023.0.2.x86_64.rpm                  1.7 MB/s |  45 kB     00:00
(6/13): generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch.rpm   626 kB/s |  19 kB     00:00
(7/13): httpd-core-2.4.69-1.amzn2023.0.2.x86_64.rpm              33 MB/s | 1.4 MB     00:00
(8/13): httpd-filesystem-2.4.69-1.amzn2023.0.2.noarch.rpm       290 kB/s |  12 kB     00:00
(9/13): httpd-tools-2.4.69-1.amzn2023.0.2.x86_64.rpm            1.9 MB/s |  79 kB     00:00
```

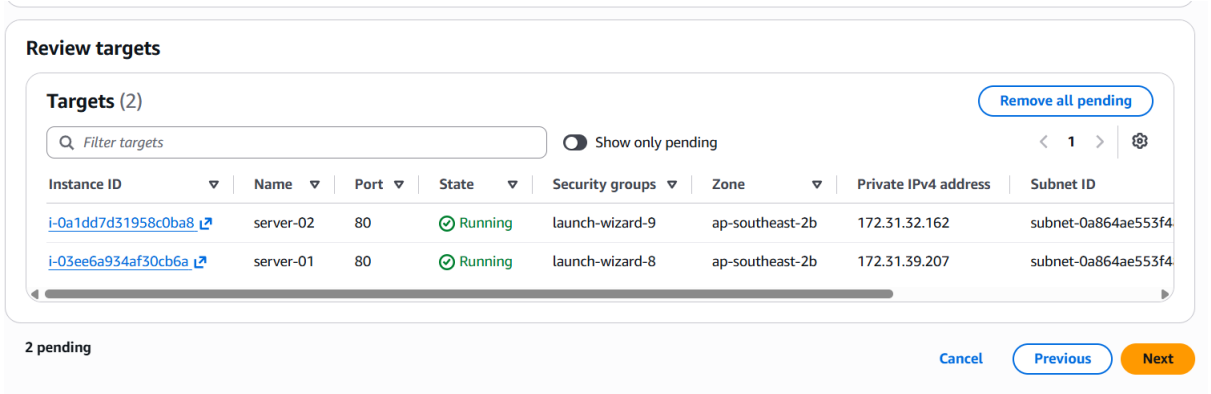
```
Complete!
[ec2-user@server1 ~]$ sudo systemctl enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
[ec2-user@server1 ~]$ sudo systemctl status httpd
○ httpd.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
     Active: inactive (dead)
       Docs: man:httpd.service(8)
[ec2-user@server1 ~]$ sudo systemctl start httpd
[ec2-user@server1 ~]$ echo "heelo from server 1" | sudo tee /var/www/html/index.html
heelo from server 1
[ec2-user@server1 ~]$ echo "Hello from Server 1" | sudo tee /var/www/html/index.html
Hello from Server 1
[ec2-user@server1 ~]$ curl http://localhost
Hello from Server 1
[ec2-user@server1 ~]$
```

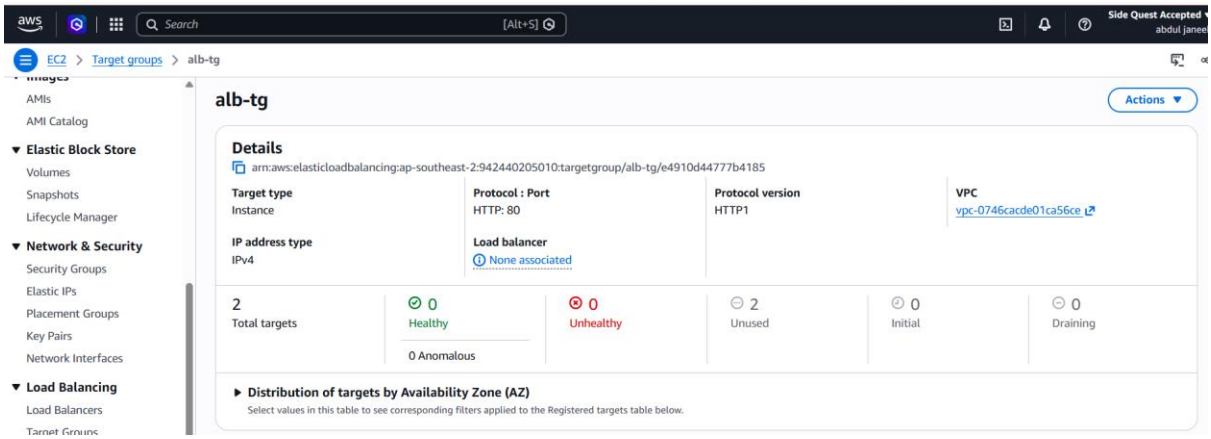
- Next create a Target Group

A Target Group is a group of servers that the load balancer sends requests to.

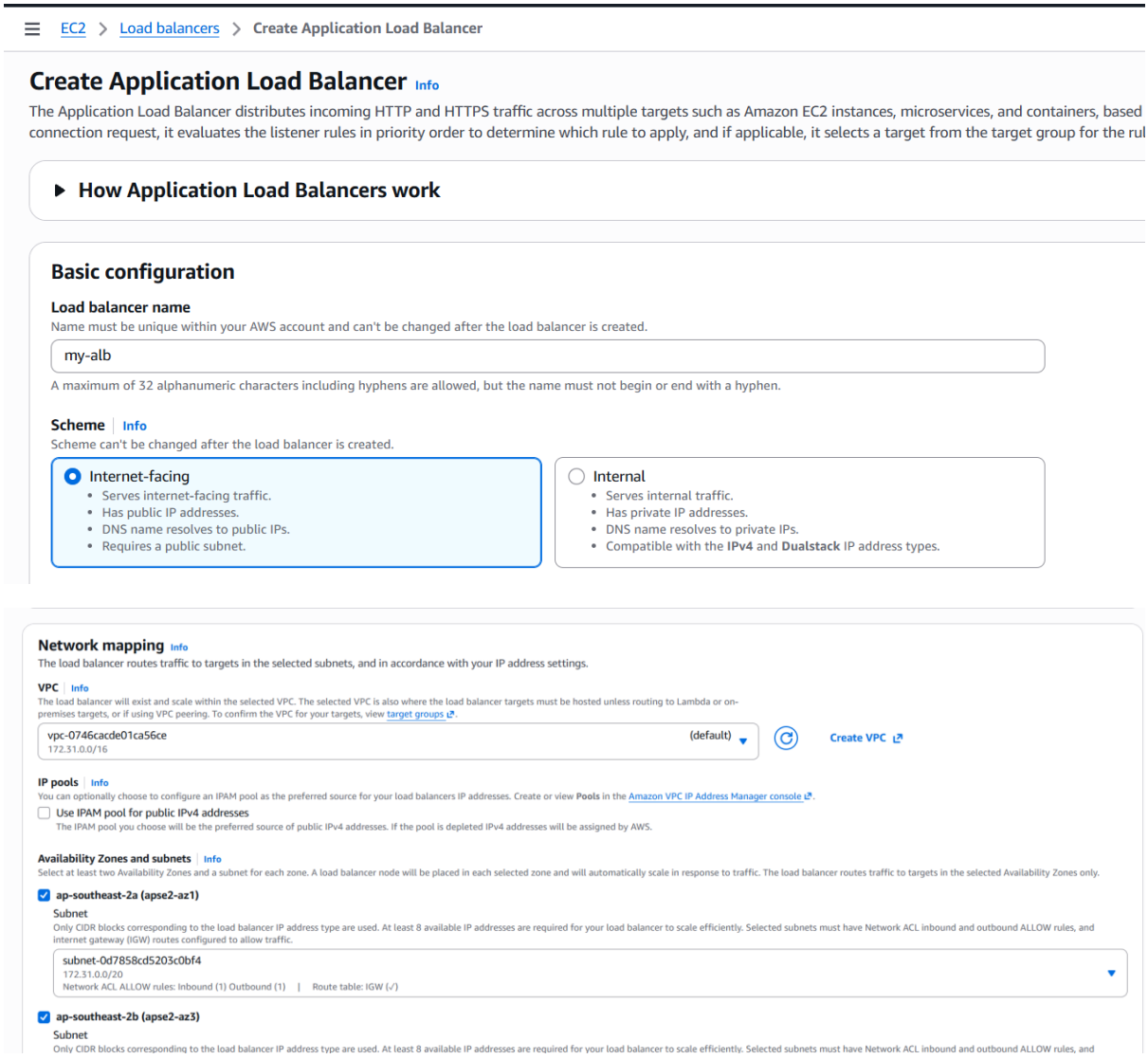




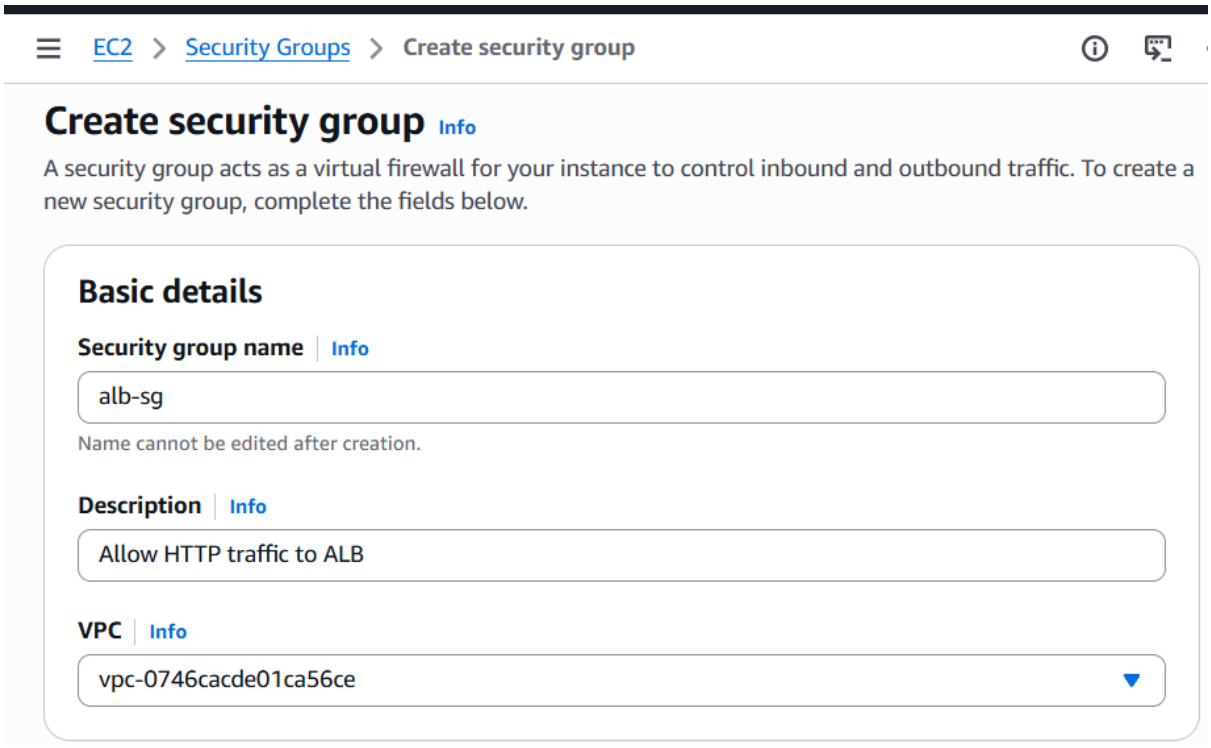


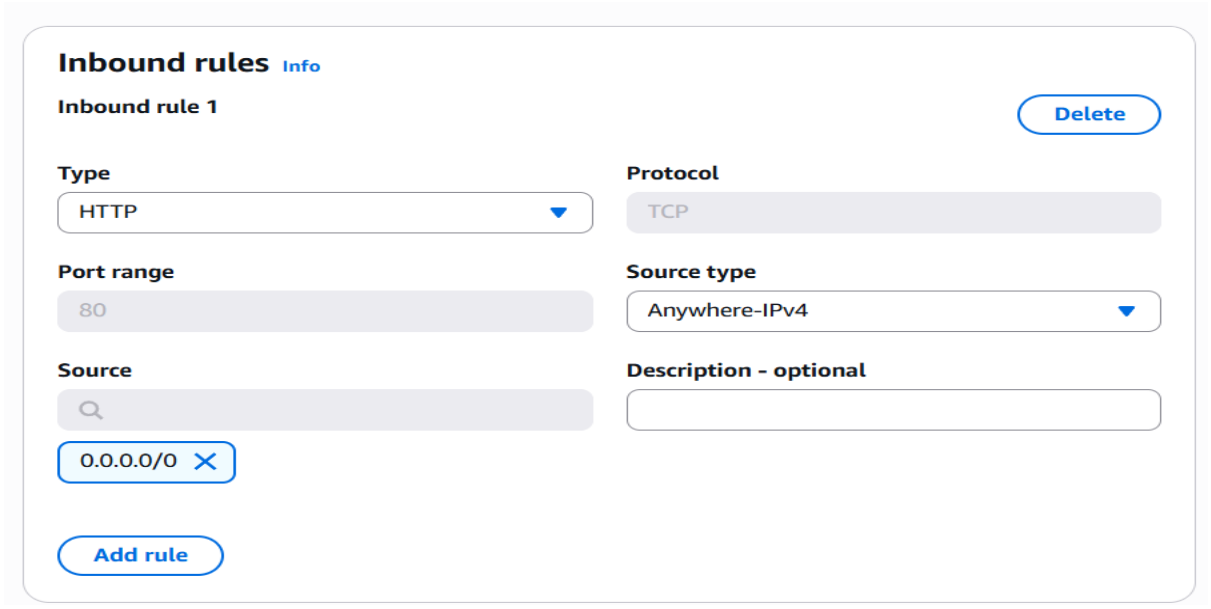


- Next step creating the application load balancer

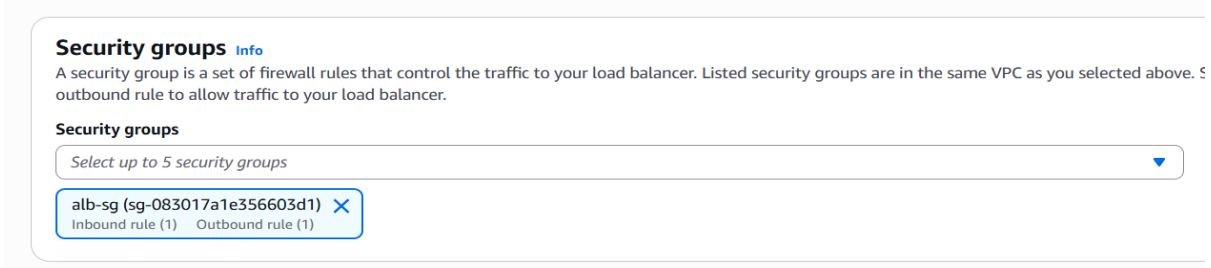


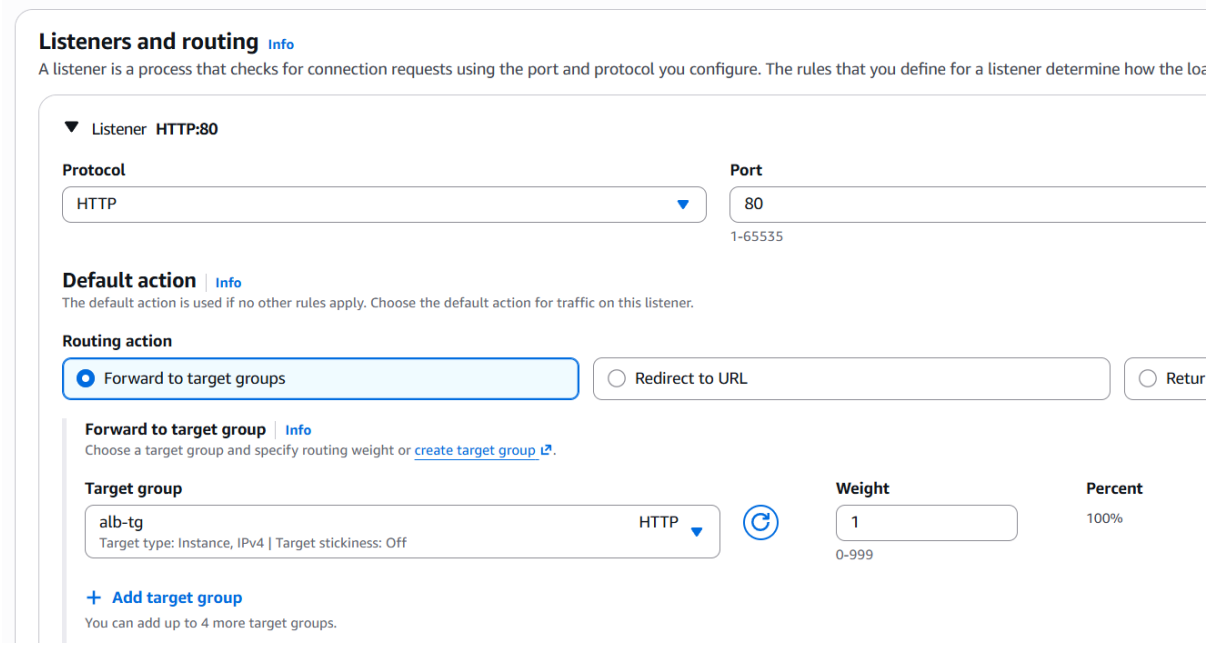
- Then created a separate security group named alb-sg for your Application Load Balancer.



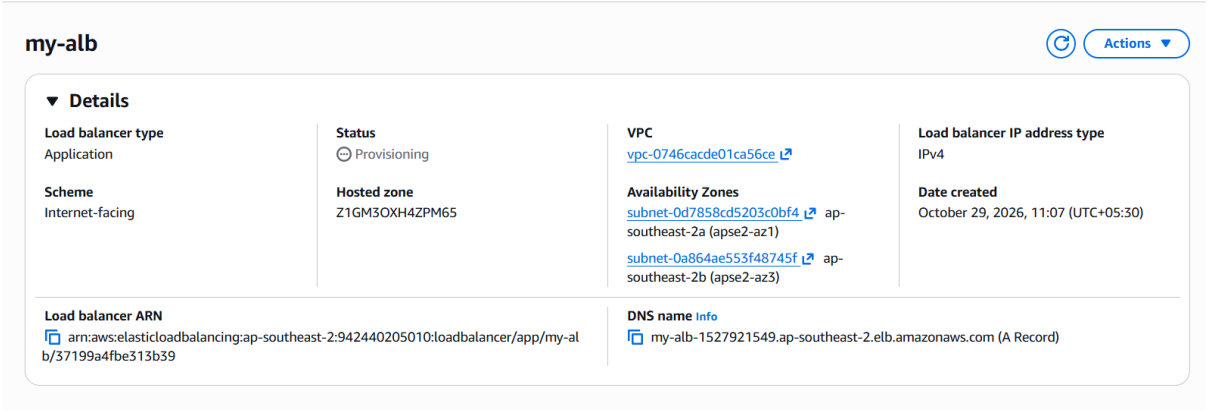


- Added that security group to ALB

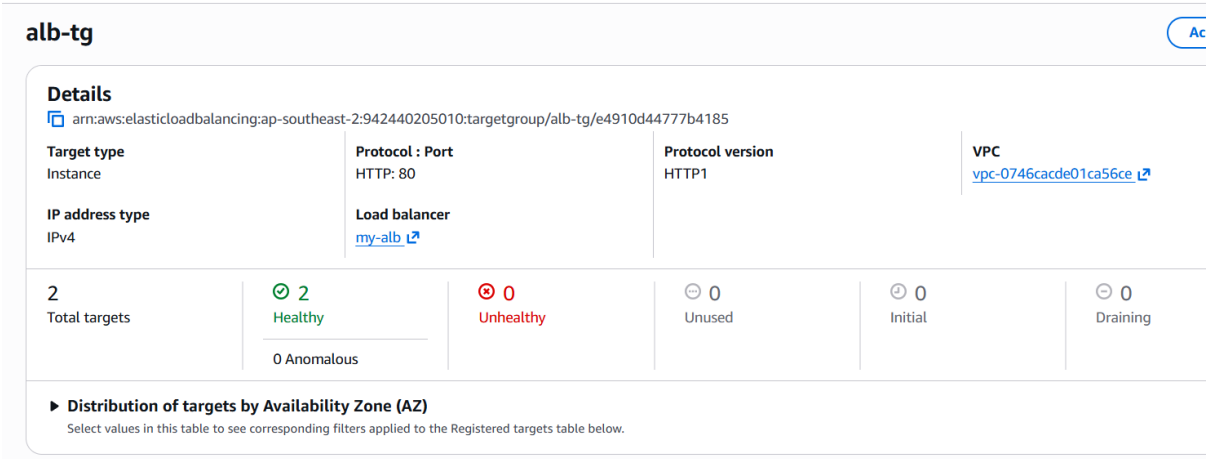




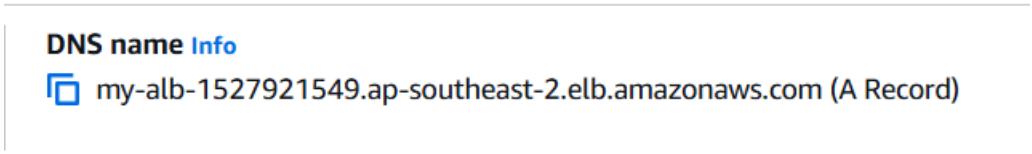
- Alb created

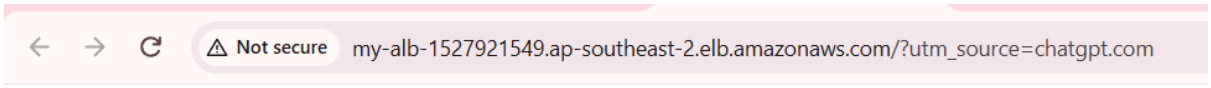


- I checked my target group added to servers is healthy

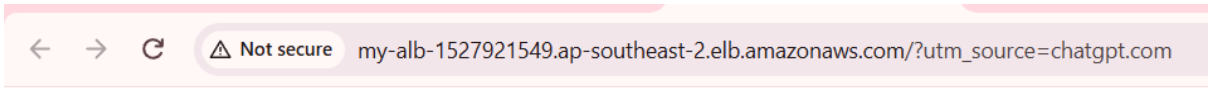


- And tested both EC2 servers through ALB and the load balancer is working correctly





Hello from Server 1



Hello from Server 2

Task 2: Health Checks

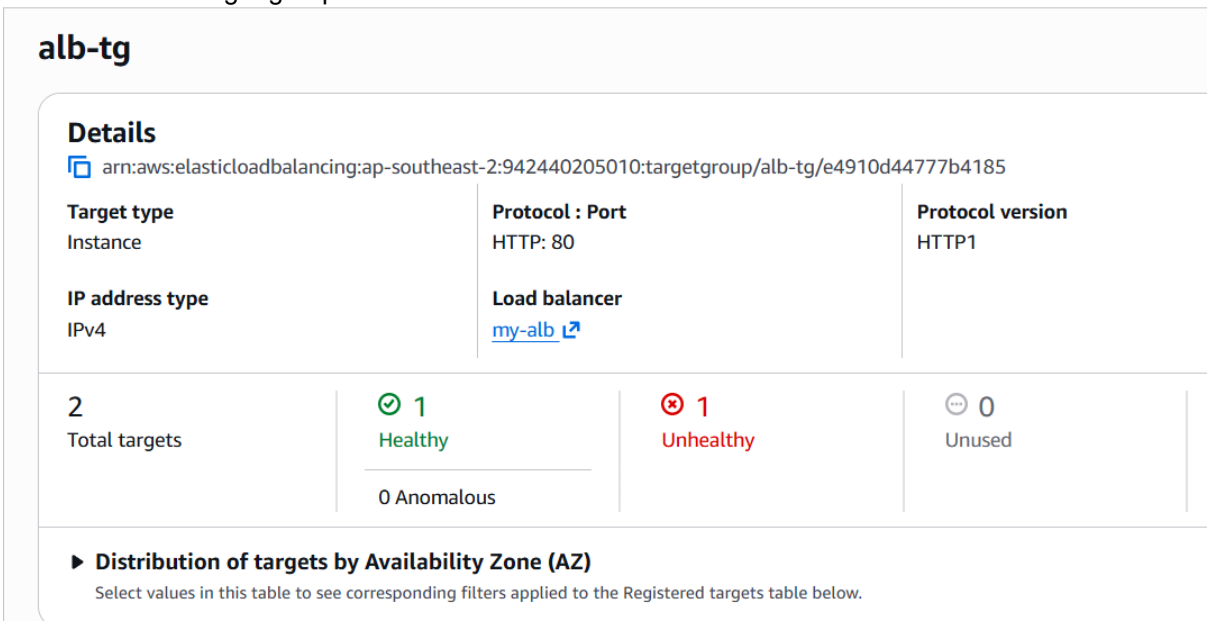
- Stop Apache on server 1

- Connected to Server 1 and stopped the Apache web server using:
- `Sudo systemctl stop httpd`

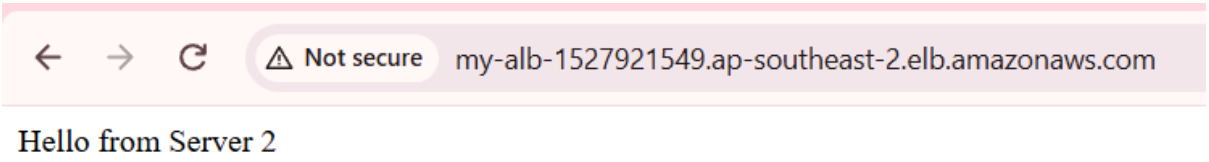
```
C:\Users\hp\Documents\awskey>ssh -i "ec2-web-key.pem" ec2-user@ec2-3-107-75-204.ap-southeast-2.compute.amazonaws.com
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Thu Oct 29 04:40:37 2026 from 103.70.197.13
[ec2-user@server1 ~]$ sudo systemctl stop httpd
[ec2-user@server1 ~]$ sudo systemctl status httpd
o httpd.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
     Active: inactive (dead) since Thu 2026-10-29 07:59:58 UTC; 10min ago
   Duration: 3h 11min 29.676s
       Docs: man:httpd.service(8)
    Process: 26587 ExecStart=/usr/sbin/httpd $OPTIONS -DFOREGROUND (code=exited, status=0/SUCCESS)
   Main PID: 26587 (code=exited, status=0/SUCCESS)
     Status: "Total requests: 679; Idle/Busy workers 100/0;Requests/sec: 0.0591; Bytes served/sec:  26 >        CPU:

Oct 29 04:48:27 server1 systemd[1]: Starting httpd.service - The Apache HTTP Server...
Oct 29 04:48:27 server1 httpd[26587]: AH00558: httpd: Could not reliably determine the server's fully q>Oct 29 04:48:
Oct 29 04:48:27 server1 httpd[26587]: Server configured, listening on: port 80
Oct 29 07:59:57 server1 systemd[1]: Stopping httpd.service - The Apache HTTP Server...
Oct 29 07:59:58 server1 systemd[1]: httpd.service: Deactivated successfully.
Oct 29 07:59:58 server1 systemd[1]: Stopped httpd.service - The Apache HTTP Server.
```

- Verified the target group Health status

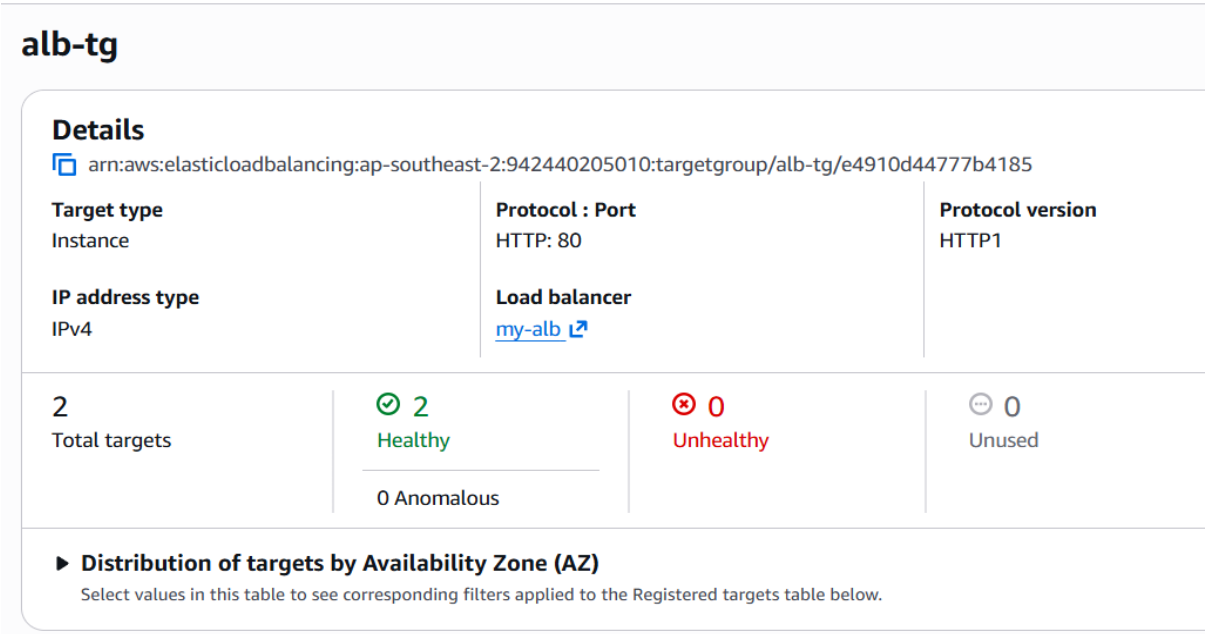


- Then tested the ALB



- This confirming that the ALB continued forwarding request to healthy server while server 1 was unhealthy.

- After that I restarted apache on server to return to HEALTHY status.task 2 is completed



- To configure path-based routing on an AWS Application Load Balancer (ALB) so that requests to different URL paths are forwarded to different target groups.
- Create Application Directories
- On Server 1, create the Application 1 directory and its web page.

```
</body></html>
[ec2-user@server1 ~]$ echo "Welcome to Application 1 - Server 1" | sudo tee /var/www/html/app1/index.html
Welcome to Application 1 - Server 1
[ec2-user@server1 ~]$ curl http://localhost/app1/
Welcome to Application 1 - Server 1
```

- On Server 2, create the Application 2 directory and its web page.

```
C:\Users\hp\Documents\awskey>ssh -i "ec2-web-key.pem" ec2-user@ec2-54-252-253-162.ap-southeast-2.compute.amazonaws.com
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Thu Oct 29 04:43:11 2026 from 103.70.197.13
[ec2-user@server2 ~]$ sudo mkdir -p /var/www/html/app2
echo "Welcome to Application 2 - Server 2" | sudo tee /var/www/html/app2/index.html[ec2-user@server2 ~]$
[ec2-user@server2 ~]$ echo "Welcome to Application 2 - Server 2" | sudo tee /var/www/html/app2/index.html
Welcome to Application 2 - Server 2
[ec2-user@server2 ~]$ curl http://localhost/app2/
Welcome to Application 2 - Server 2
```

- Then Created two target groups in the AWS EC2 console.

Target Group 1

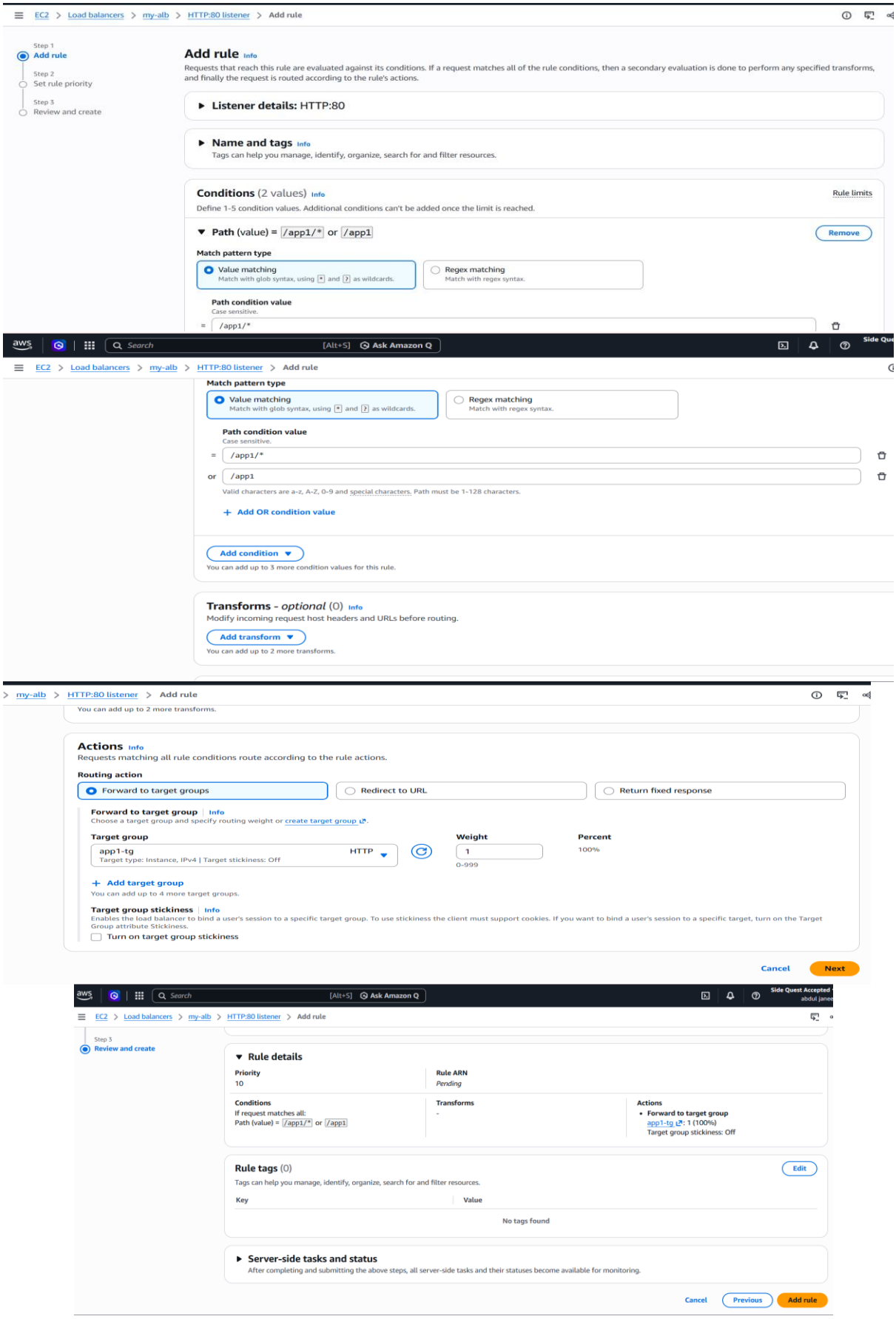
- Name: app1-tg
- Target type: Instances
- Protocol: HTTP
- Port: 80
- Registered target: Server 1
- Health check path: /app1/

Target Group 2

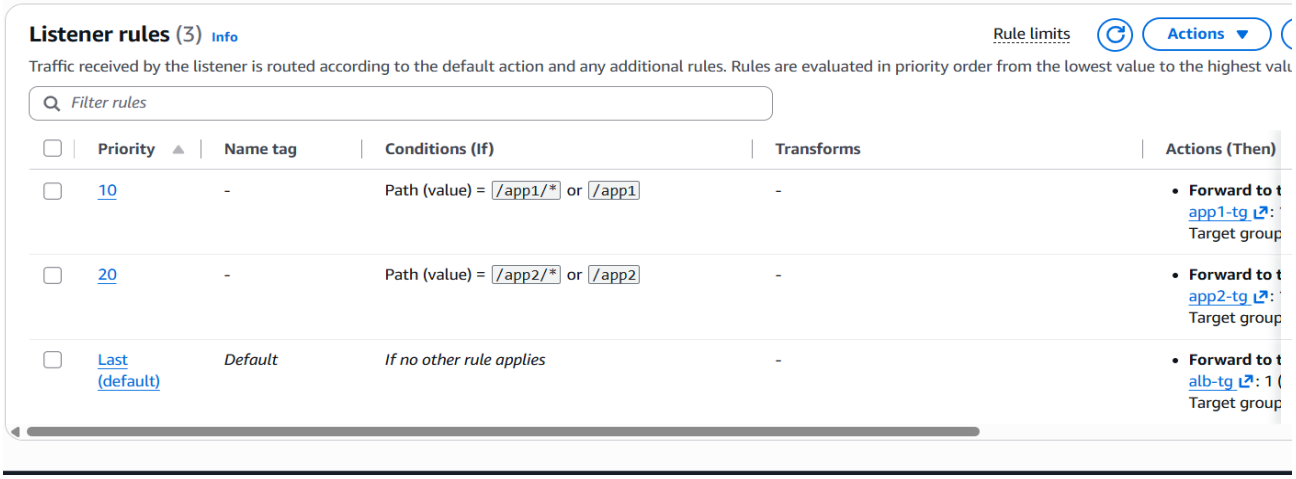
- Name: app2-tg
- Target type: Instances
- Protocol: HTTP
- Port: 80
- Registered target: Server 2
- Health check path: /app2/

Both target groups use the same VPC as the existing ALB.

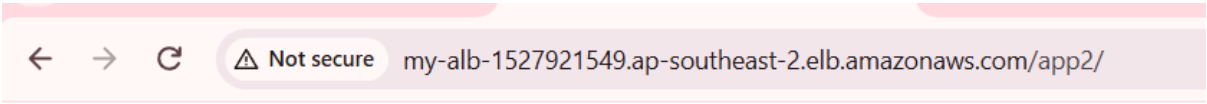
- Configuring ALB Listener Rules



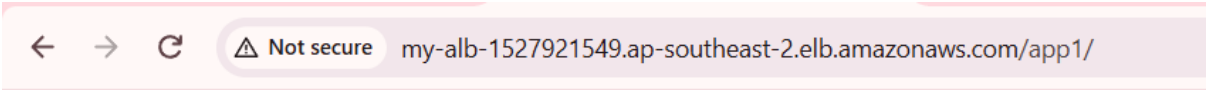
- This like adding app2 also



- Used the ALB DNS name to test both applications.



Welcome to Application 2 - Server 2



Welcome to Application 1 - Server 1

- **Host-Based Routing**
- Using two hostnames and configure listener rules so each hostname routes to a different target group.
- Since we don't own a registered domain, we're using the ALB-provided DNS name and its resolved IP address for our temporary hostname testing.

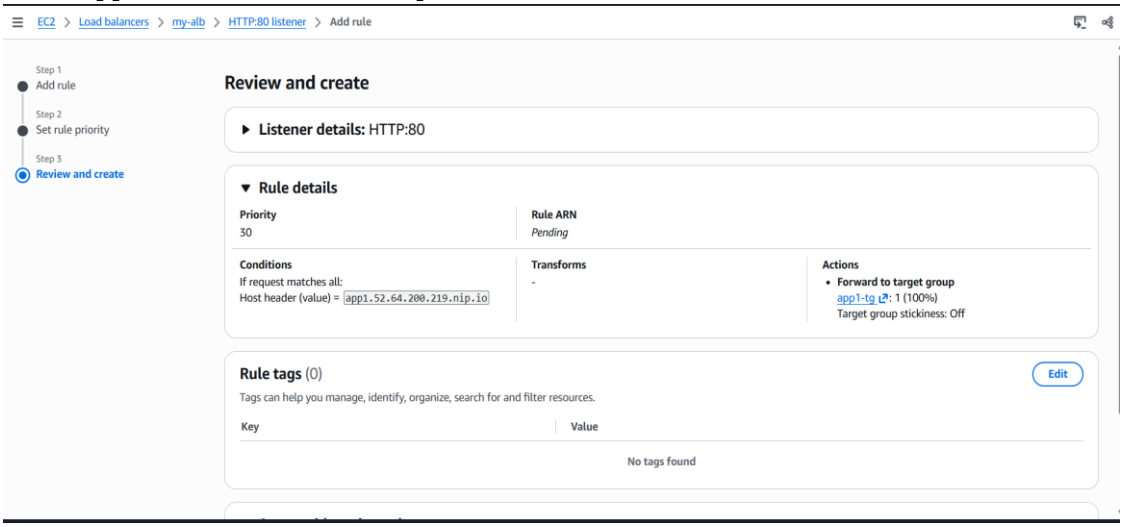
```
C:\Users\hp>nslookup my-alb-1527921549.ap-southeast-2.elb.amazonaws.com
Server:  dns.keralavisionisp.com
Address:  103.199.160.80

Non-authoritative answer:
Name:    my-alb-1527921549.ap-southeast-2.elb.amazonaws.com
Addresses:  52.64.200.219
          16.176.124.85


C:\Users\hp>_
```

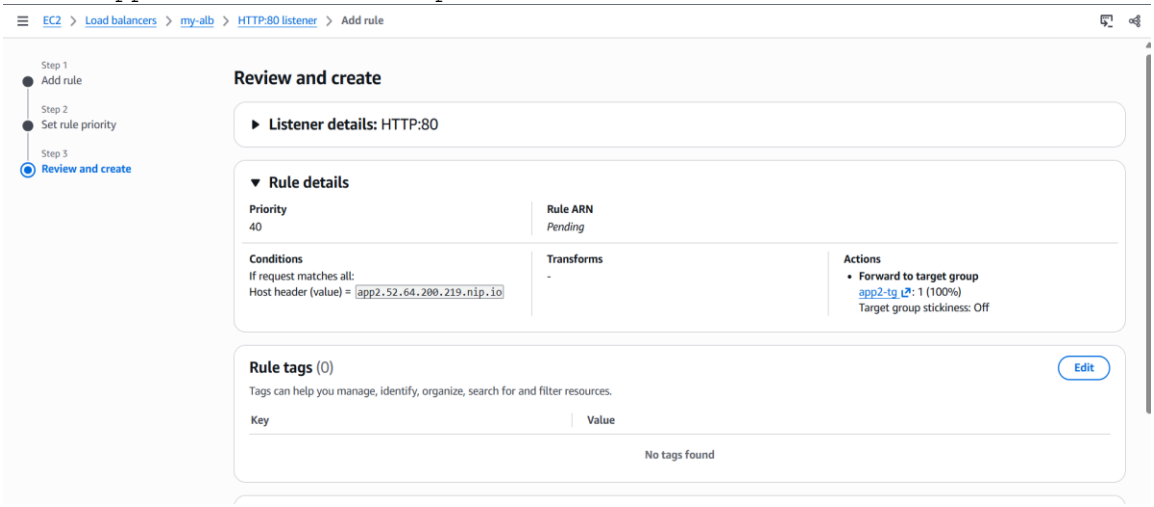
- After that configuring listner rule
- For Application 1

- `app1.52.64.200.219.nip.io`



- For application 2

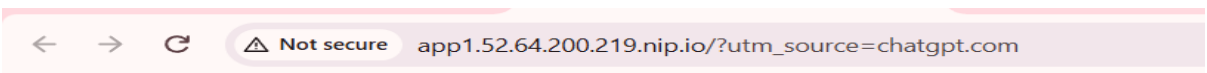
- `app2.52.64.200.219.nip.io`



- after that successfully tested Host-Based Routing

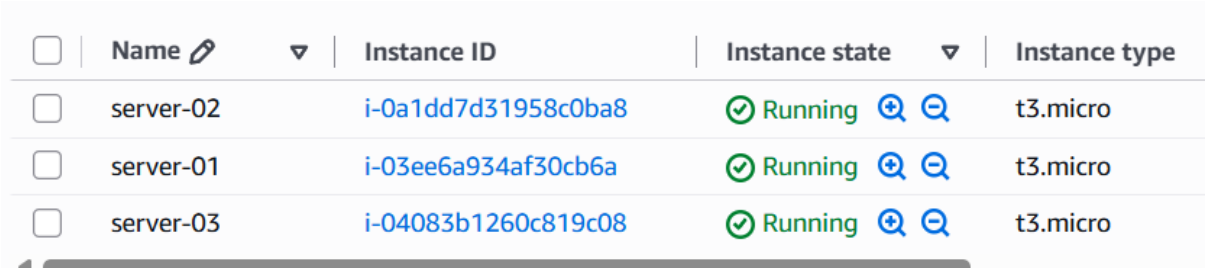


Hello from Server 1

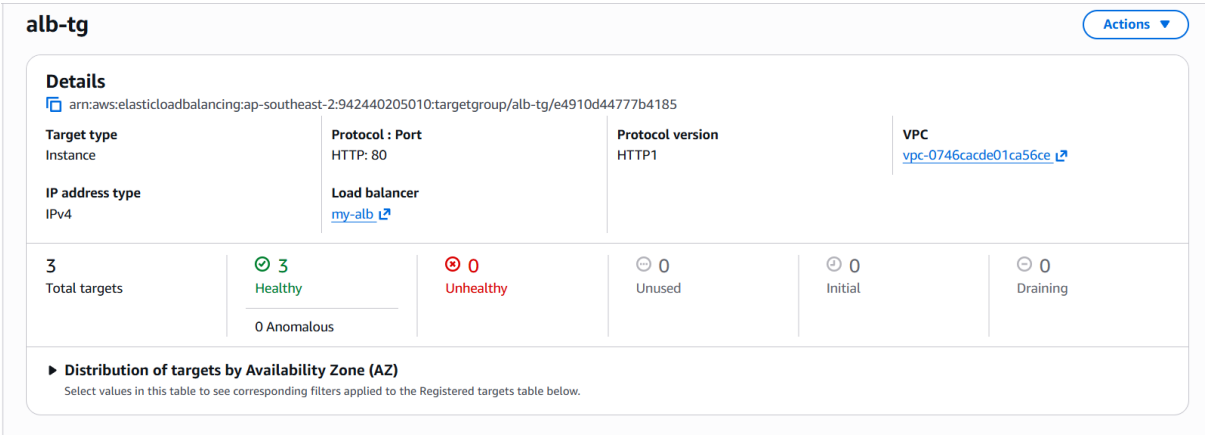


Hello from Server 1

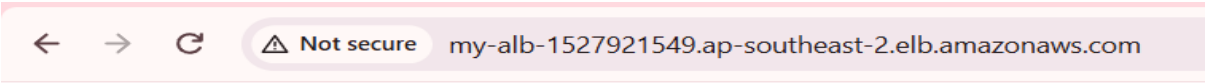
- **Launching a third EC2 instance, install the application, add it to the target group and verify traffic reaches all three instances.**



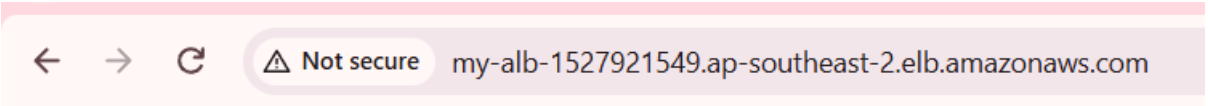
Name	Instance ID	Instance state	Instance type
server-02	i-0a1dd7d31958c0ba8	Running	t3.micro
server-01	i-03ee6a934af30cb6a	Running	t3.micro
server-03	i-04083b1260c819c08	Running	t3.micro



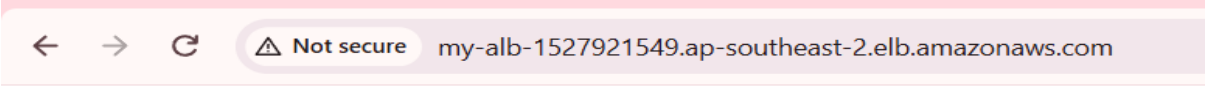
- All three servers are successfully receiving traffic



Hello from Server 1

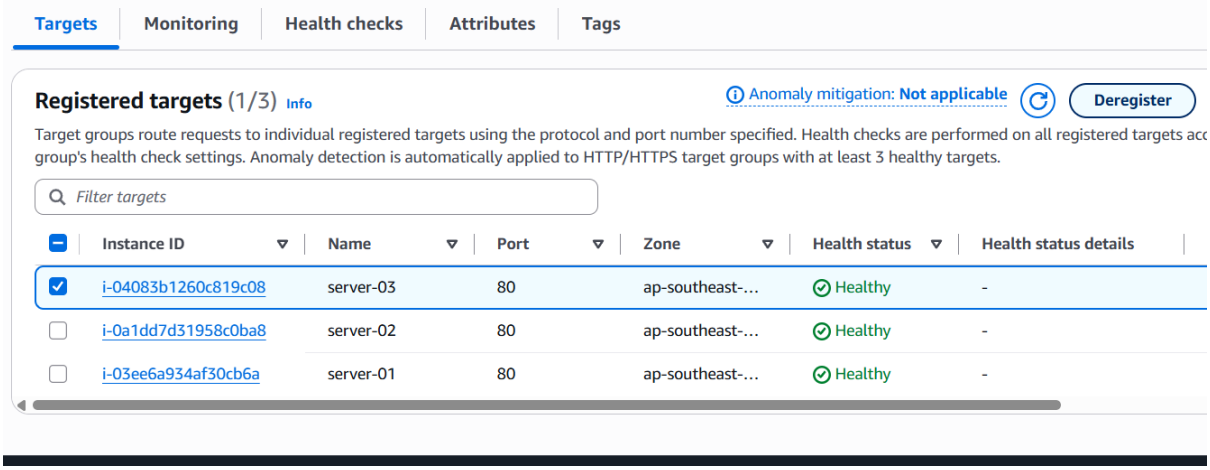


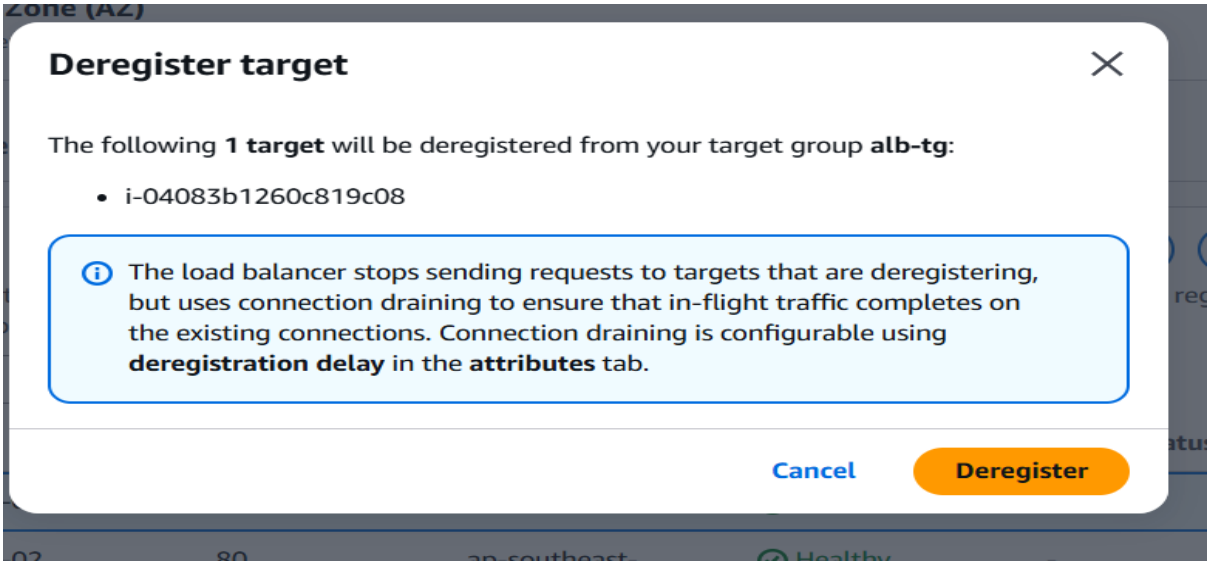
Hello from Server 2

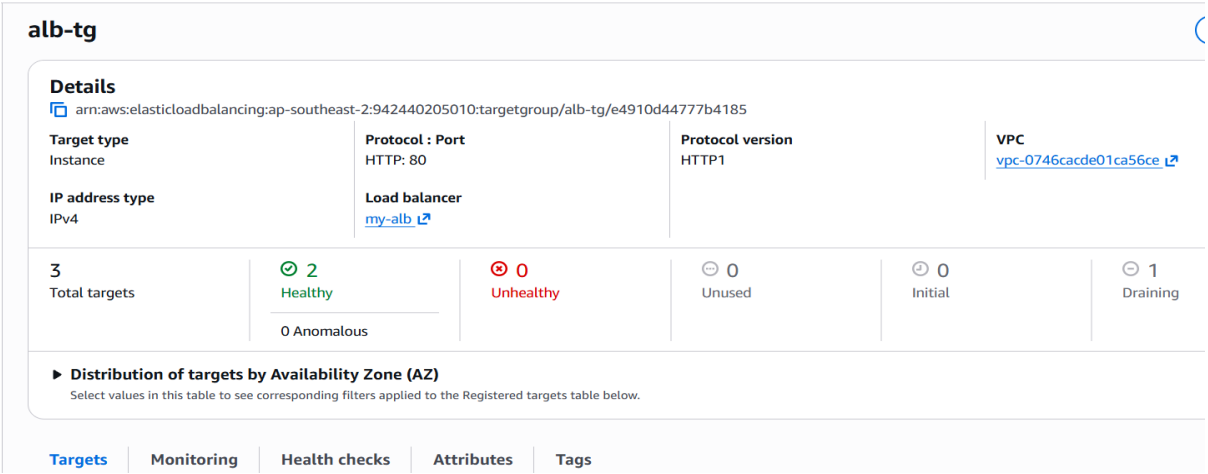


This is Server 3

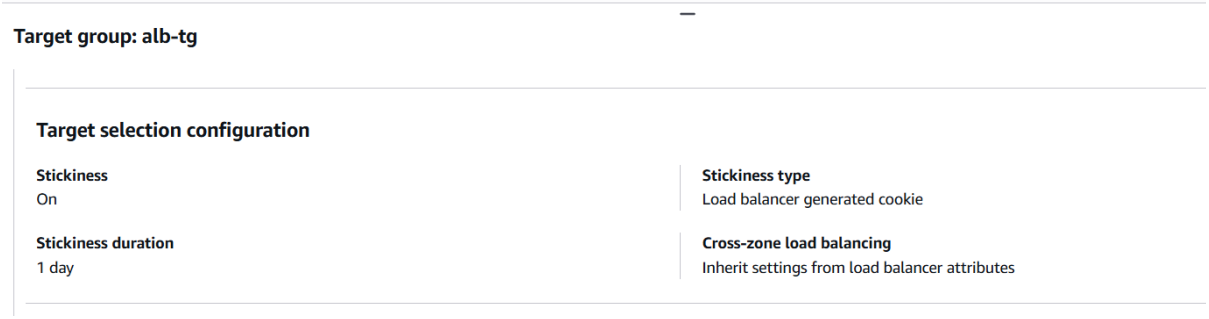
- **Deregister Instance**
- Removing one instance from the target group and verifying traffic is only served by the remaining instances.



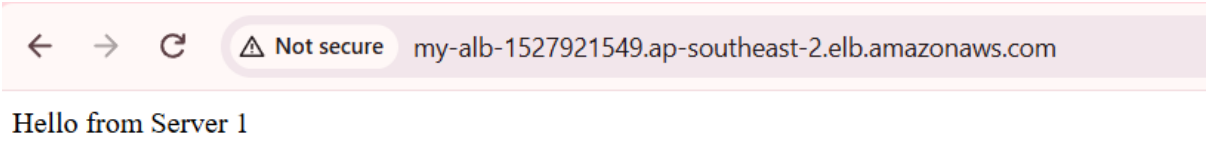




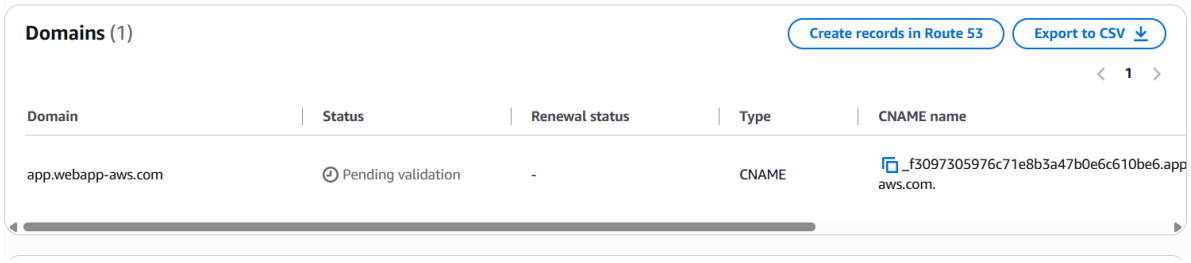
- **Sticky Sessions**
- Enable stickiness on the target group and verify repeated requests from the same browser reach the same backend.



- After enable sticky only get the same backend



- **HTTPS**
- Create an ACM certificate, configure HTTPS listener on the ALB and redirect HTTP to HTTPS.



Requested a public SSL/TLS certificate through AWS Certificate Manager for app.webapp-aws.com. The certificate remained in Pending validation because a registered domain was not available to complete DNS ownership verification. Therefore, the certificate could not be issued, and HTTPS listener configuration was not completed.

- **Multi-Port Routing**
- Run applications on different ports and configure target groups and listener rules accordingly.
- To run applications on different ports and configure target groups and ALB listener rules to route traffic to the correct backend servers.
- First I checked port 8080 is available or not
- Sudo ss -lntp | grep :8080
- Thrn changed 80 to 8080 throuch edit conf file this done in server 3

- Then configured SELinux and firewall on server 3

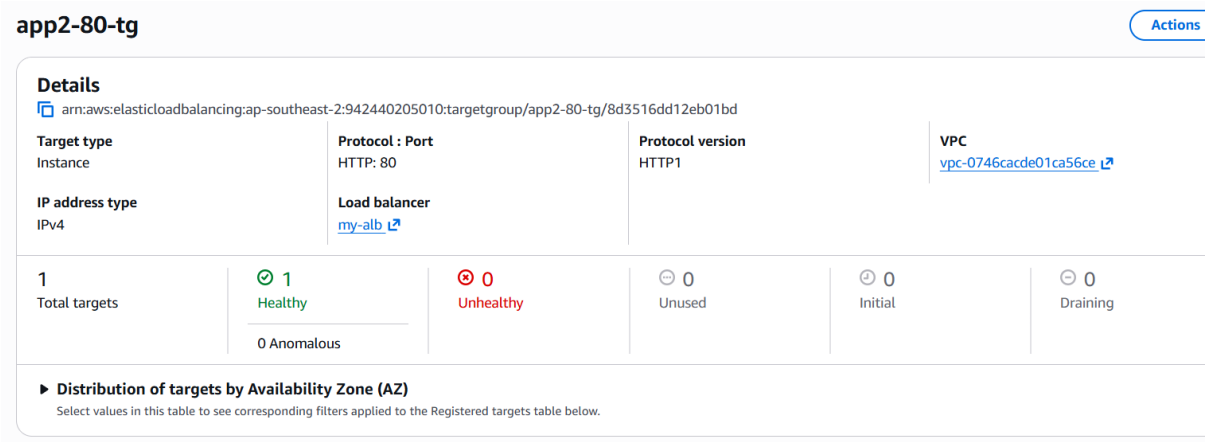
- `sudo semanage port -a -t http_port_t -p tcp 8080`
- Allow port 8080 through the firewall:
- `sudo firewall-cmd --permanent --add-port=8080/tcp`
- `sudo firewall-cmd --reload`
- Restart Apache:
- `sudo systemctl restart httpd`
- `sudo systemctl status httpd`
- Verify the listening port:
- `sudo ss -lntp | grep :8080`
- Then created webpage on both servers

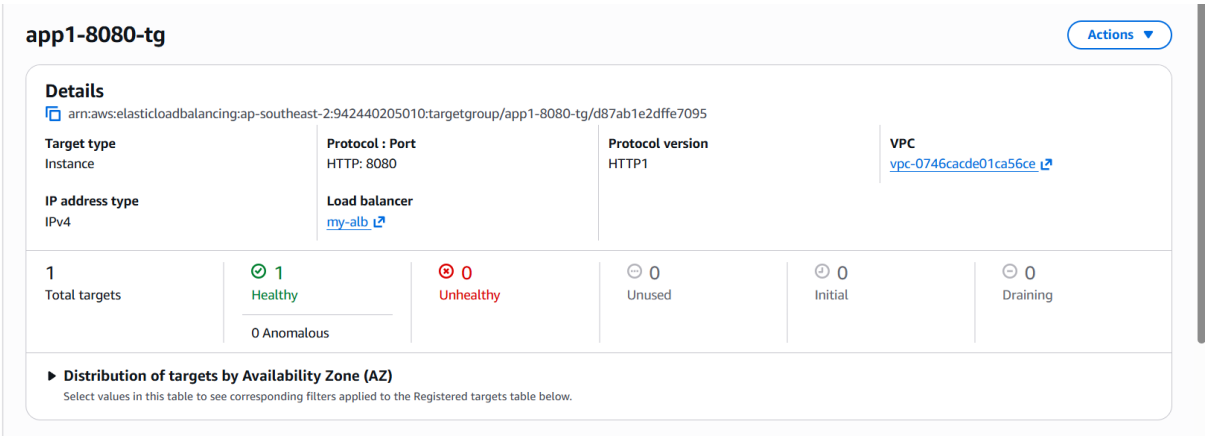
```
[ec2-user@server3 ~]$ sudo ss -lntp | grep 8080
LISTEN 0      511                *:8080            *:*    users:(("httpd",pid=1875,fd=4),("httpd",pid=1784,fd=4),("httpd",pid=1783,fd=4),("httpd",pid=1778,fd=4))
[ec2-user@server3 ~]$ curl http://localhost:8080
This is Server 3
[ec2-user@server3 ~]$  sudo mkdir -p /var/www/html/app8080
[ec2-user@server3 ~]$ echo "Welcome to Application 1 - Server 3 (Port 8080)" | sudo tee /var/www/html/app8080/index.html
Welcome to Application 1 - Server 3 (Port 8080)
[ec2-user@server3 ~]$ curl http://localhost:8080/app8080/
Welcome to Application 1 - Server 3 (Port 8080)
[ec2-user@server3 ~]$
```

```
C:\Users\hp\Documents\awskey>ssh -i "ec2-web-key.pem" ec2-user@ec2-54-252-253-162.ap-southeast-2.compute.amazonaws.com

A newer release of "Amazon Linux" is available.
  Version 2023.12.20261029:
Run "/usr/bin/dnf check-release-update" for full release and version update info
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Thu Oct 29 08:35:03 2026 from 103.70.197.13
[ec2-user@server2 ~]$ sudo mkdir -p /var/www/html/app80
[ec2-user@server2 ~]$ echo "Welcome to Application 2 - Server 2 (Port 80)" | sudo tee /var/www/html/app80/index.html
Welcome to Application 2 - Server 2 (Port 80)
[ec2-user@server2 ~]$
```

- Updated the EC2 security group inbound rules to allow traffic from the ALB security group.
- After that created 2 target groups

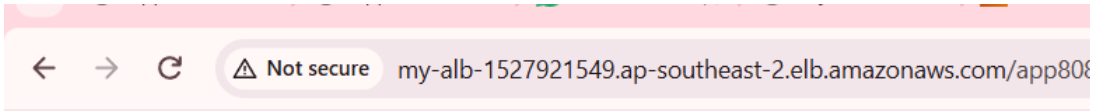


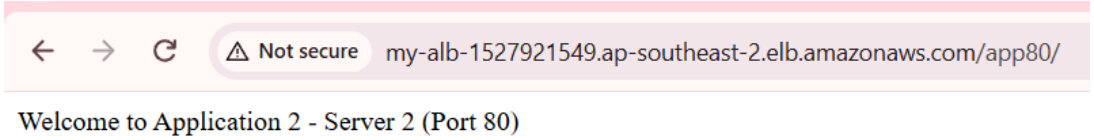


- Then configured ALB Listner Rules

Priority	Path condition	Forward to
50	/app8080/*	app1-8080-tg
60	/app80/*	app2-80-tg

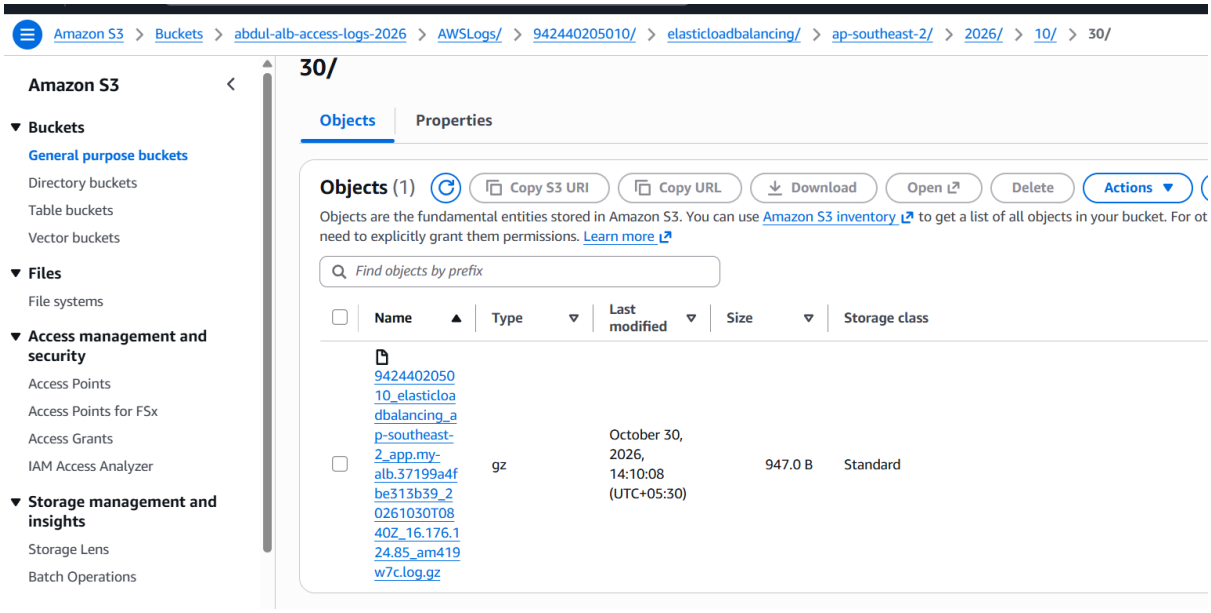
- Tested multi-porting routing





- Successfully configured multi-port routing using an AWS Application Load Balancer. Two applications running on different backend ports (8080 and 80) were accessed through separate ALB listener rules.

- **Access Logs**
- Enable ALB access logs to an S3 bucket, generate traffic and inspect the log files.
- **Steps Performed**
- Created an S3 bucket named abdul-alb-access-logs-2026.
- Added an S3 bucket policy to allow the ELB log delivery service to upload log files.
- Opened EC2 → Load Balancers → my-alb → Attributes.
- Enabled Access Logs and selected the S3 bucket.
- Generated traffic by accessing the ALB DNS several times.
- Opened the S3 bucket and verified the AWSLogs folder.
- Found the compressed .log.gz access log files.

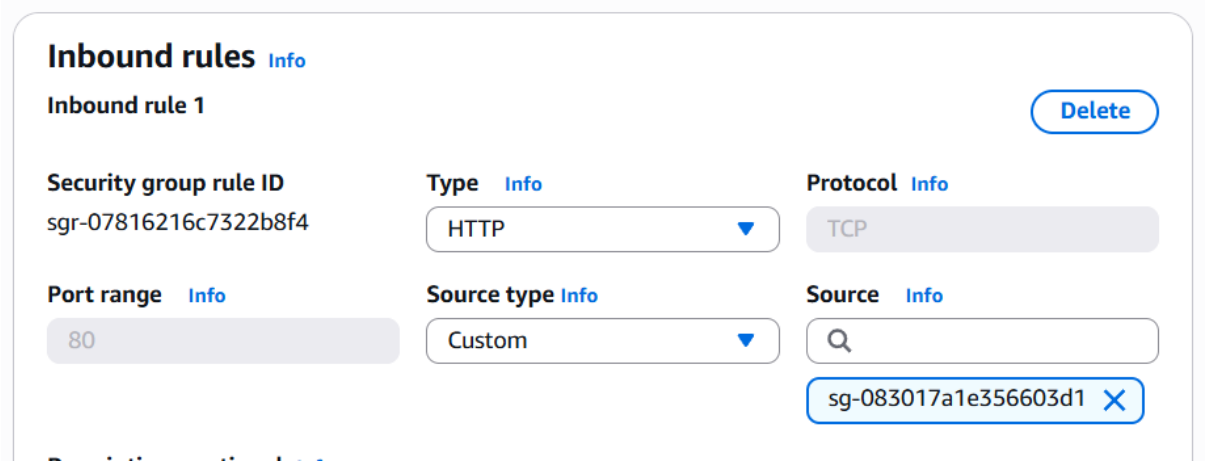


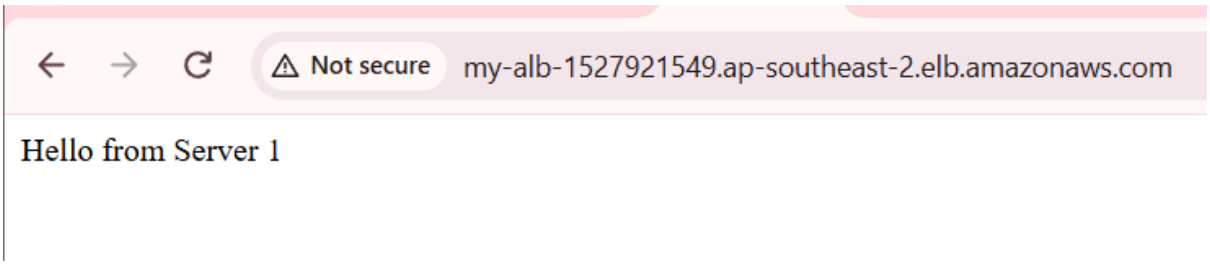
Successfully enabled ALB access logs and verified that log files were stored in the S3 bucket.

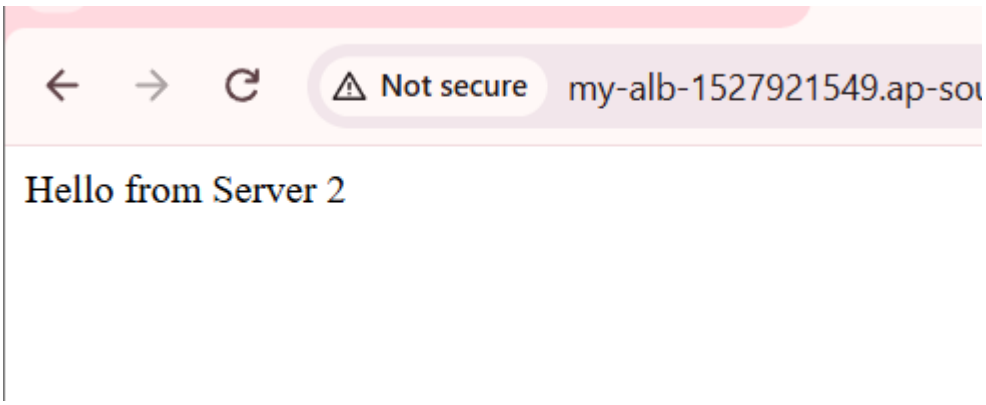
- **Security Groups**
- Allow internet traffic only to the ALB and allow EC2 instances to accept traffic only from the ALB security group.

Steps Performed

1. Checked the security group attached to the ALB (alb-sg).
2. Configured the ALB security group to allow public HTTP traffic on port 80.
3. Updated the EC2 security groups for Server 1, Server 2 and Server 3.
4. Allowed HTTP traffic from the ALB security group instead of allowing direct public access.
5. Preserved SSH access for server administration.
6. Verified that all backend servers became healthy in their target groups.
7. Tested website access through the ALB DNS.







- **Auto Scaling**
- Create a Launch Template and Auto Scaling Group attached to the ALB target group. Increase desired capacity and verify new instances are automatically registered.

- Created a Launch Template named `ALB-Web-LT` with the required AMI, instance type, key pair, and backend security group. Configured User Data to automatically install Apache, enable and start the HTTPD service, and create a sample web page. Created an Auto Scaling Group named `ALB-Web-ASG` and attached it to the ALB Target Group `alb-tg`. Set the desired capacity to 3 instances and updated the group to use the latest Launch Template version. Finally, started an Instance Refresh to replace the old instances with new instances and monitored the refresh progress and target health.

